

# GNN Testing Report

Propagation-confidence GNN, NSE Auto supply-chain graph · generated 03 Sep 2026 · suite: 58 passed, 0 failed, 1 skipped

## 1. What is being tested

The GNN is one of the three models in the trading system. It looks at the 15-company supply-chain graph (32 curated supplier-to-buyer links) and, for each directed pair of companies  $A \rightarrow B$ , estimates the probability that a *shock* at A today will be followed by a shock at B within the next five trading days. A shock is an unusually large one-day price move. These probabilities are computed once for every trading day, cached to disk, and fed into the reinforcement-learning agent's state. The RL agent never calls the GNN directly.

Testing therefore has to answer four questions: (1) are the training labels built correctly, (2) is the data reaching the model in the right shape with no leakage from the future, (3) does the model architecture behave the way a graph model should, and (4) does the trained checkpoint actually contain signal that a trivial rule of thumb would not give. Section 3 explains each test in plain words; Section 2 analyses what the trained model does.

## 2. Results analysis

### 2.1 Headline numbers

| Metric                                                | Value  |
|-------------------------------------------------------|--------|
| Validation AUC (trained checkpoint, recomputed)       | 0.6100 |
| Validation AUC recorded when training finished        | 0.6100 |
| Training AUC                                          | 0.6885 |
| Train – val gap                                       | 0.0785 |
| Baseline: train-period return correlation of the pair | 0.5784 |
| Baseline: buyer's previous-day volatility             | 0.3523 |
| Baseline: supplier's previous-day volatility          | 0.4231 |
| Validation positive rate (a coin-flip scores 0.5 AUC) | 22.6%  |
| Early-stopping epoch                                  | 5      |

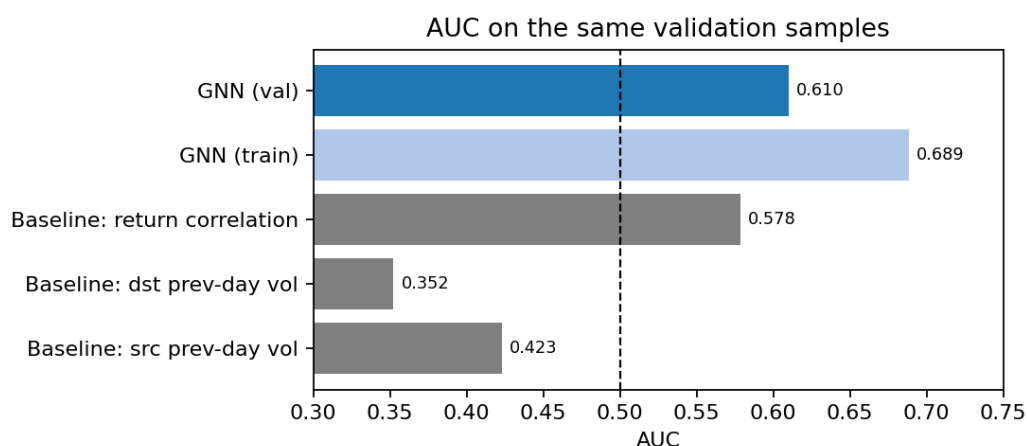


Figure 1 — AUC of the GNN and three no-learning baselines on the identical 571 validation samples.

AUC (area under the ROC curve) measures ranking quality: 1.0 means every real cascade is scored above every non-cascade, 0.5 means random. The GNN scores 0.610. The strongest baseline — simply looking up how correlated the two stocks' returns were during the training period — scores 0.578. So the GNN adds +0.032 AUC on top of "pairs that co-move tend to cascade". That margin is real but thin. The two volatility baselines score *below* 0.5, meaning that in this validation window the more volatile stocks cascaded less often; the model has not learned a lazy "volatile stock = shock" shortcut, which is what the volatility-scaled shock definition was designed to prevent.

## 2.2 Data under test

| Item               | Value                                                                      |
|--------------------|----------------------------------------------------------------------------|
| Timeline           | 1174 trading days, 2021-10-01 -> 2026-06-30                                |
| Graph              | 15 companies, 32 curated edges, 64 scored directed pairs (both directions) |
| Node features      | logret_1, logret_5, logret_21, vol_21, mom_63, volz_21 (6)                 |
| Training samples   | 2133 (17.8% positive), 2021-10-06 -> 2025-05-15                            |
| Validation samples | 571 (22.6% positive), 2025-07-03 -> 2026-06-17                             |
| Shock-day rate     | 3.61% of all (day, company) cells                                          |
| Model              | graphsage, hidden 64, 2 layers, dropout 0.4, GRU over 10 days, seed 42     |

Validation is the chronological tail of the timeline with a five-day gap before it, so no training label window overlaps the validation period. The per-company shock counts are even (37–51 shock days each over 4.7 years), confirming the volatility-scaled threshold gives every stock a comparable number of events.

## 2.3 Is the signal stable over time?

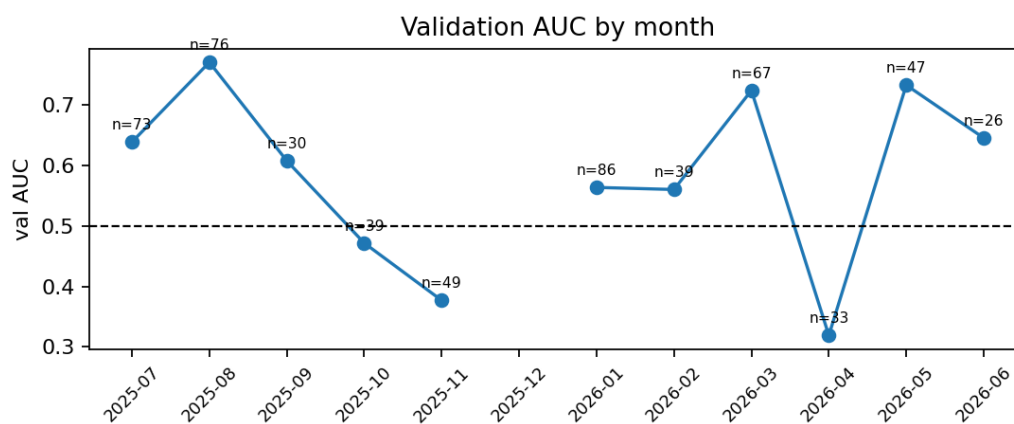


Figure 2 — validation AUC computed month by month (n = samples in that month).

No. The month-by-month AUC ranges from 0.32 (2026-04) to 0.77 (2025-08), and 3 of 11 months fall below 0.5 (2025-10, 2025-11, 2026-04). Monthly sample sizes are small (6–90) so a lot of this is noise, but the practical reading is that the GNN's score is an intermittent signal, not a steady one. The RL agent has to learn how much to trust it, which is exactly what the 'no-gnn' ablation arm will measure.

## 2.4 Direction, calibration and features

| Direction                     | n   | positive rate | AUC   |
|-------------------------------|-----|---------------|-------|
| Downstream (supplier → buyer) | 272 | 19.1%         | 0.622 |
| Upstream (buyer → supplier)   | 299 | 25.8%         | 0.630 |

Both directions score about equally, which justifies the decision to supervise both (supply disruptions travel downstream to the car maker; demand shocks travel upstream to the supplier). Neither half is dead weight.

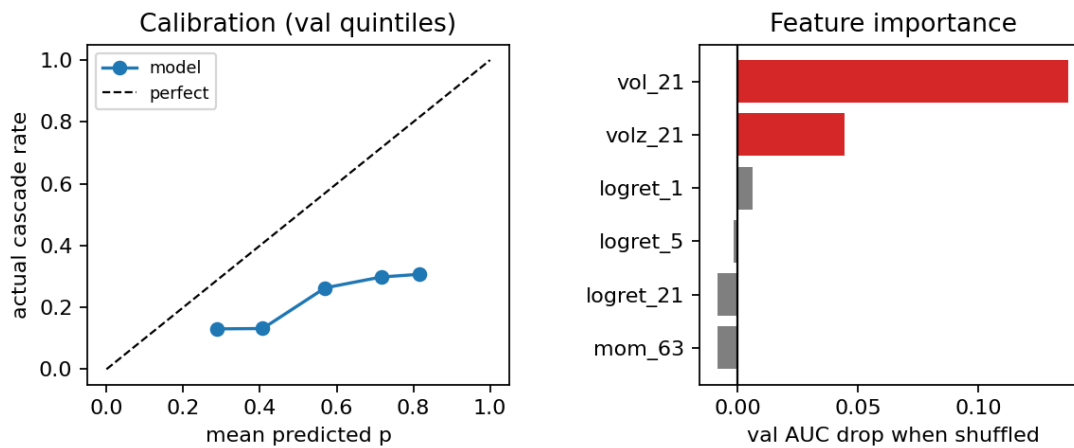


Figure 3 (left) — calibration: mean predicted probability per quintile vs actual cascade rate. Figure 4 (right) — permutation importance: validation AUC drop when one input feature is shuffled across days.

**Calibration.** The five quintiles are ordered correctly: the lowest-scored fifth of pairs cascaded 13% of the time, the highest-scored fifth 31%. The absolute probabilities are inflated (mean predicted 0.56 against a 23% base rate) because training re-weights the rare positive class; this is intended. The score is a ranking, not a calibrated probability, and the RL state treats it as such.

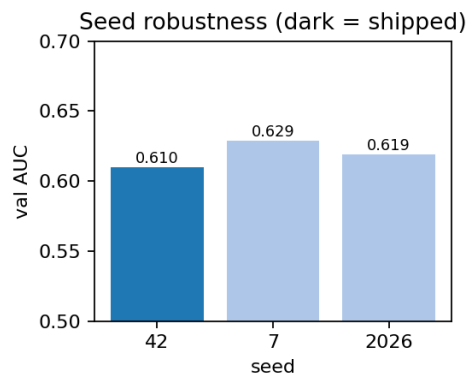
**Features.** Shuffling **vol\_21** costs 0.138 AUC — almost the entire signal. **volz\_21** (volume z-score) adds a little. The four return and momentum features contribute nothing measurable: the model is essentially answering "is the supplier in a high-volatility regime, and is volume unusual?" This is a candidate for later feature work (e.g. sector-relative returns), but it is not a defect — it is what the data supports.

## 2.5 Per-pair view

| Pair                        | n | pos | AUC  |
|-----------------------------|---|-----|------|
| BHARATFORG.NS->EICHERMOT.NS | 9 | 2   | 1.00 |
| BHARATFORG.NS->M&M.NS       | 9 | 2   | 1.00 |
| EXIDEIND.NS->TMPV.NS        | 9 | 1   | 1.00 |
| EXIDEIND.NS->EICHERMOT.NS   | 9 | 4   | 0.95 |
| ...                         |   |     |      |
| BOSCHLTD.NS->TMPV.NS        | 9 | 2   | 0.36 |
| HEROMOTOCO.NS->BOSCHLTD.NS  | 9 | 2   | 0.36 |
| BOSCHLTD.NS->EICHERMOT.NS   | 9 | 2   | 0.29 |
| BOSCHLTD.NS->MARUTI.NS      | 9 | 4   | 0.20 |

36 pairs have enough validation samples to score; 7 are below 0.5. The weak pairs cluster around **BOSCHLTD.NS** (6 of them). With roughly nine samples per pair this is a hint rather than a verdict, but it points at that node's curated edges in relationships.csv as the first thing to re-examine.

## 2.6 Seed robustness



Retraining from scratch with two other random seeds gives 0.610–0.629. The shipped seed (42) ranks 3 of 3; it also stopped earliest (epoch 5). The spread across seeds (0.019) is larger than the margin over the correlation baseline, so the headline AUC should be quoted as "about 0.61–0.63", not to three decimals. Retrains went to a temporary folder; the shipped weights were not touched.

## 2.7 Reproducibility and leakage

The cached score file used by the RL agent (1174 days  $\times$  64 pairs) was recomputed from the weights: maximum difference 0.0e+00, i.e. bit-for-bit identical. Corrupting every day *after* day  $t$  with noise leaves day  $t$ 's scores exactly unchanged, while nudging a day *inside* the ten-day window does change them. Together these prove the scores the agent sees are deterministic and contain no information from the future. The train/val gap of 0.079 with early stopping at epoch 5 shows no overfitting.

## 3. The tests, in plain words

The suite lives in `tests/gnn/` and runs with `pytest tests/gnn`. Each test is a small Python function that sets up a situation where the correct answer is known in advance, runs the real project code on it, and asserts the answer matches. If any assertion fails, `pytest` reports it. There are 59 tests in five groups. Tests in groups A–D use tiny hand-made inputs and need nothing on disk; group E uses the real data and the real trained weights and skips itself if those are missing.

### A. Shock detection and label construction (13 tests)

**What is checked.** A "shock" is defined as a one-day move larger than  $2.5\times$  the stock's recent (21-day) volatility, with a floor of 2% so that calm periods don't flag trivial moves. A training example is "company A shocked today; did company B shock within the next five trading days?" These tests confirm that definition is implemented exactly.

**How.** Each test builds a miniature price table for two fake companies (a few days, numbers chosen by hand) and calls the same shock and label functions the training code uses. For example: with 1% volatility yesterday, a 3% move must count as a shock and a 2% move must not; a 50% move on the very first day must *not* count, because there is no previous-day volatility to compare against; the volatility used must be yesterday's, not today's, so a big move cannot raise its own threshold. For labels: a shock at B on the same day as A is not a cascade; a shock exactly five days later is; six days later is not; shocks in the last five days of the timeline are dropped because their five-day window would be cut off. The train/validation split is checked to be strictly chronological with a five-day gap, and never shuffled.

### B. Data-to-tensor bridge (13 tests)

**What is checked.** The feature table has to be turned into a 3-D array of [day, company, feature] in a fixed company order, and the model is fed a trailing ten-day window ending at the day being scored. Any mistake here silently corrupts everything downstream.

**How.** Small tables with recognisable numbers (1, 2, 3 for company A; 10, 20, 30 for company B) are converted and specific cells are checked. The windowing function is asked for a ten-day window at day 1 and must return two real days preceded by zero-padding, never a day after the target. On the real dataset the tests check the array shape ( $1174 \times 15 \times 6$ ), that every curated edge has its reverse present for message passing, that the 64 scored pairs are exactly the 32 edges plus their reverses with no duplicates, that features were normalised using

only training-period statistics (mean  $\approx 0$ , std  $\approx 1$  on training rows), and that the training/validation counts match the numbers recorded at training time.

### C. Model architecture (14 tests)

**What is checked.** Properties any correct graph neural network must have, independent of data. These catch wiring bugs that would not necessarily crash but would make the model meaningless.

**How.** The model is built on random inputs and probed. Two models built with the same seed must give identical outputs (reproducibility). In evaluation mode two calls must agree exactly; in training mode they must differ (dropout is really on/off). After one backward pass every parameter must have a non-zero gradient (nothing is disconnected). Scoring four pairs together must equal scoring them one at a time (no cross-talk).  $A \rightarrow B$  and  $B \rightarrow A$  must get different scores (direction is respected). Renumbering the companies consistently in the features, the edges and the pairs must not change any score — this is the defining property of a graph network, called permutation equivariance. Reversing the order of the ten days must change the output (the recurrent layer is really order-aware), and deleting all edges must change it (the graph is really used). The alternative attention architecture builds and runs; an unknown architecture name is rejected; the parameter count equals the number the configuration implies.

### D. Training utilities and a learning smoke test (7 tests)

**What is checked.** The AUC metric, the per-day batching, the class re-weighting, and — most importantly — that the full training loop can actually learn.

**How.** The AUC implementation is fed cases with known answers: perfectly ordered scores must give 1.0, perfectly reversed 0.0, all-equal 0.5, a single class must give "undefined", and on 500 random points it must agree with scikit-learn's reference to six decimals (skipped if scikit-learn is not installed). The batching helper is checked to group samples by day in date order. The class weight is checked to make one positive and one negative example carry equal total loss mass. The smoke test then builds a synthetic 8-company ring graph, plants a simple rule ("cascade if the supplier's first feature is positive"), and trains the real model for 60 steps: the loss must at least halve and the AUC must exceed 0.9. If gradients, batching or shapes were broken, this would fail.

### E. The trained checkpoint (12 tests, real data)

**What is checked.** That the weights on disk are the ones described by their metadata, that they reproduce the recorded score, that they beat naive baselines, and that the cached scores consumed by the RL agent are exact and leak-free.

**How.** The metadata file is compared field by field to the configuration. Every tensor in the saved weights is checked for shape and for finite values. The saved normalisation statistics are compared to those recomputed from the data. The validation AUC is recomputed and must match the recorded 0.610 within 0.005. The two baselines are computed on exactly the same validation samples and the GNN must beat both. The train/val gap must be under 0.2 and the scores must have spread (a constant output would pass everything else). The cache file must have the right columns, length, date index, no missing values and all values in  $[0, 1]$ , and five sampled days are recomputed from the weights and must match to five decimals. Finally the no-look-ahead test: score day 300, overwrite all later days with noise, score again — identical; nudge one day inside the window — different.

### F. Diagnostics (not pass/fail)

The report script adds analyses that have no single right answer and are therefore reported rather than asserted: the baselines above; AUC split by direction and by month; calibration by quintile; permutation importance (shuffle one feature's values across days, re-score, measure the AUC drop); per-pair AUC; and seed robustness (retrain from scratch with different random seeds into a temporary folder and compare).

## 4. Conclusion

- **The code is correct.** All 58 tests pass. Labels, data plumbing, model mechanics and the training loop behave exactly as specified, and the checkpoint on disk reproduces its recorded metrics.
- **The pipeline is leak-free and reproducible.** Scores contain no future information and the cache is bit-identical to a fresh forward pass. This is the property that matters most for the downstream RL claim, and it is proven, not assumed.
- **The signal is real but modest.** Validation AUC  $\approx 0.61$  against 0.58 for a static correlation lookup and 0.5 for chance. It is driven almost entirely by the supplier's volatility regime. Quoted honestly: the GNN adds a few points of ranking power over "these two stocks usually move together".
- **The signal is intermittent.** Monthly AUC swings between roughly 0.3 and 0.8. The RL agent should be expected to learn a modest, not dominant, reliance on it; the 'no-gnn' ablation will quantify that.
- **The headline number is seed-sensitive at the  $\pm 0.01$  level.** Other seeds reach 0.62–0.63. The shipped seed 42 is kept for reproducibility, per the project's fixed-seed rule; report the AUC as a range.
- **Next steps if time allows** (none block PPO training): re-examine the curated edges around the weakest node; try sector-relative return features since raw returns are unused; re-run the seed sweep after any feature change.

Sources: tests/gnn/ (suite), scripts/gnn\_test\_report.py (diagnostics, also writes GNN\_TEST\_REPORT.md), scripts/gnn\_test\_pdf.py (this document), notebooks/gnn\_testing.ipynb (interactive version). pytest summary: 58 passed, 1 skipped in 4.36s.